

ECE 385
Spring 2025
Experiment #5

Simple Computer SLC-3

Rohan Shah, Anish Nagar
rohan20, anishn4
Friday, March 14, 2025, 12:10 pm
Dongming Liu, dl35@illinois.edu

1. Introduction

a. Summarize the basic functionality of the SLC-3 processor.

The SLC-3 processor is a simplified version of the LC-3 computer, which we have utilized in previous classes. This is a 16-bit processor with a 16-bit program counter, which we call the PC. The processor has three primary methods: fetch, decode, and execute. This computer will perform various operations, which we define through an Instruction Register (IR), called opcodes. This tells us what operation we want to accomplish on each opcode, we can perform ADD, AND, NOT, BRANCH, JUMP, JSR, LDR, STR, and PAUSE, and these will be explained later on. The fetch state is utilized first to read the memory address for the instruction and then figure out what instruction to decode. From here, our program counter gets incremented to point to the next instruction as we progress through the stages. The Decode state allows us to store the value of the opcode into an instruction decoder, which we then use to perform our operation in the "Execute" method. The processor contains several components, from a control unit and register unit to several multiplexers and signals, which help us identify the functionality of the overarching modules.

b. Describe how the SLC-3 differs from the LC-3 processor you learned in ECE 120/220.

The SLC-3 in Experiment 5 is a simplified version of the LC-3 processor we learned about in ECE 120/200. In this version, we do not have TRAP operations, such as OUT, PUTS, GETC, IN, and HALT. These are utilized in LC-3 to input characters, end the program, and output values. However, we do not utilize them in SLC-3, which is a primary difference. Furthermore, this SLC-3 processor incorporates pause states, which we utilize in our Finite State Machine, along with displaying the output to the Hex Display. Many of the states are reduced, including the Indirect Load (LDI), STI, LD, LEA, and several others, which help simplify our logic quite a bit.

2. Written Description and Diagrams of SLC-3

a. Summary of Operation

The basic operation of the SLC-3 computer is following the fetch, decode, and execute cycle. Fetch will take an instruction from memory. Decode figures out which instruction that is. Then the execute operation executes the instruction. The CPU then fetches the next instruction.

b. Describe in words how the SLC-3 performs its functions. You should describe the Fetch-Decode-Execute cycle as well as the various instructions the

processor can perform.

The SLC-3 processor computes all its functions with the fetch, decode, and execute cycle.

The fetch cycle is responsible for getting instructions from the memory. The PC value is used to go into the memory address register to read the instruction. Then we take the contents in MAR and put them into MDR (Memory Data Register). Then this goes into IR which is the instruction register holding the instruction to decode. Finally, the PC gets updated to PC+1.

Decode determines what type of instruction we need to execute. This happens as the instruction goes through the instruction sequencer/decoder. From the opcode, we now know what to execute.

Finally, execute uses all the internal architecture to run the instruction. Then the computer fetches the next instruction. The various instructions the processor can perform include Add, AND, NOT, Branch, JMP, JSR, LDR, STR, and Pause.

c. Block diagram of CPU.sv, which includes the main data path and control unit for your CPU.

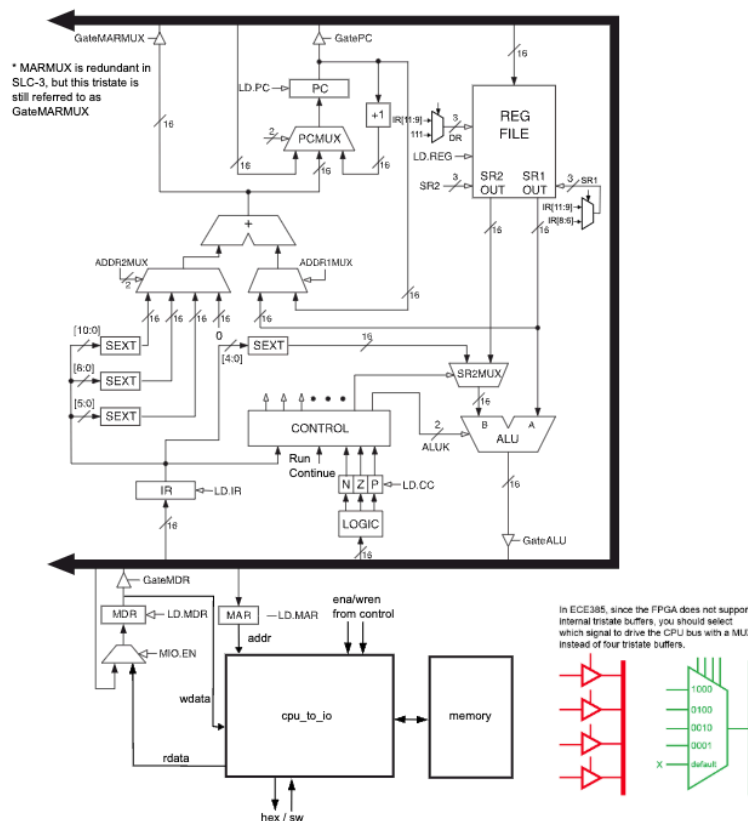


Figure 1 - block diagram of the internal CPU architecture showing the main data path and control unit

ii. CPU.SV Top Level

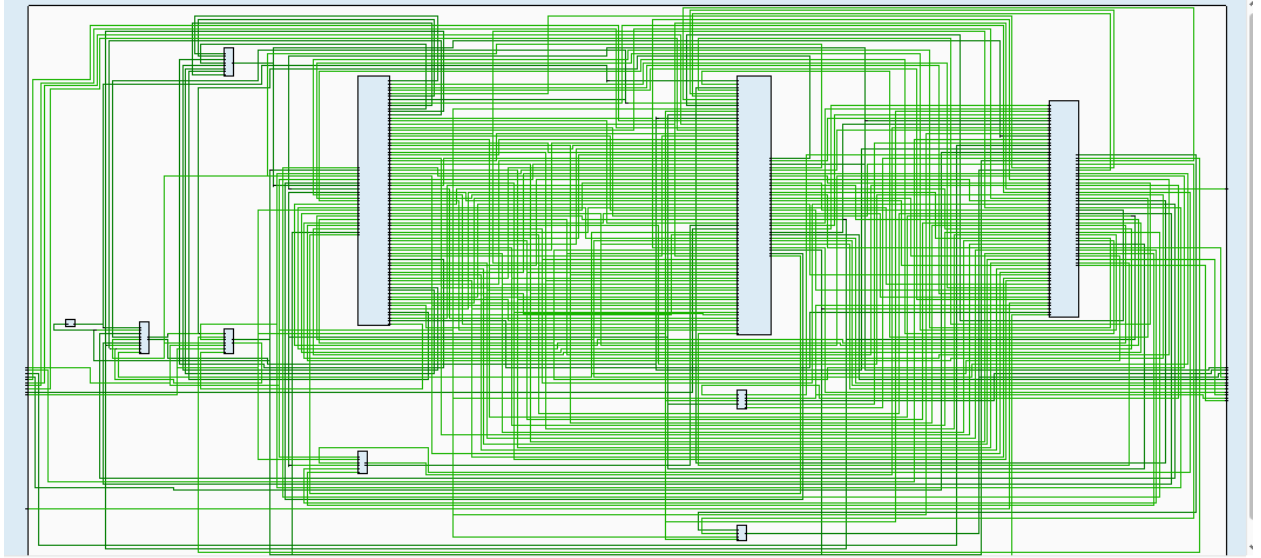


Figure 2 - CPU.sv schematic showing all internal parts of the CPU

d. Execution State Annotations

(Starting Next Page)

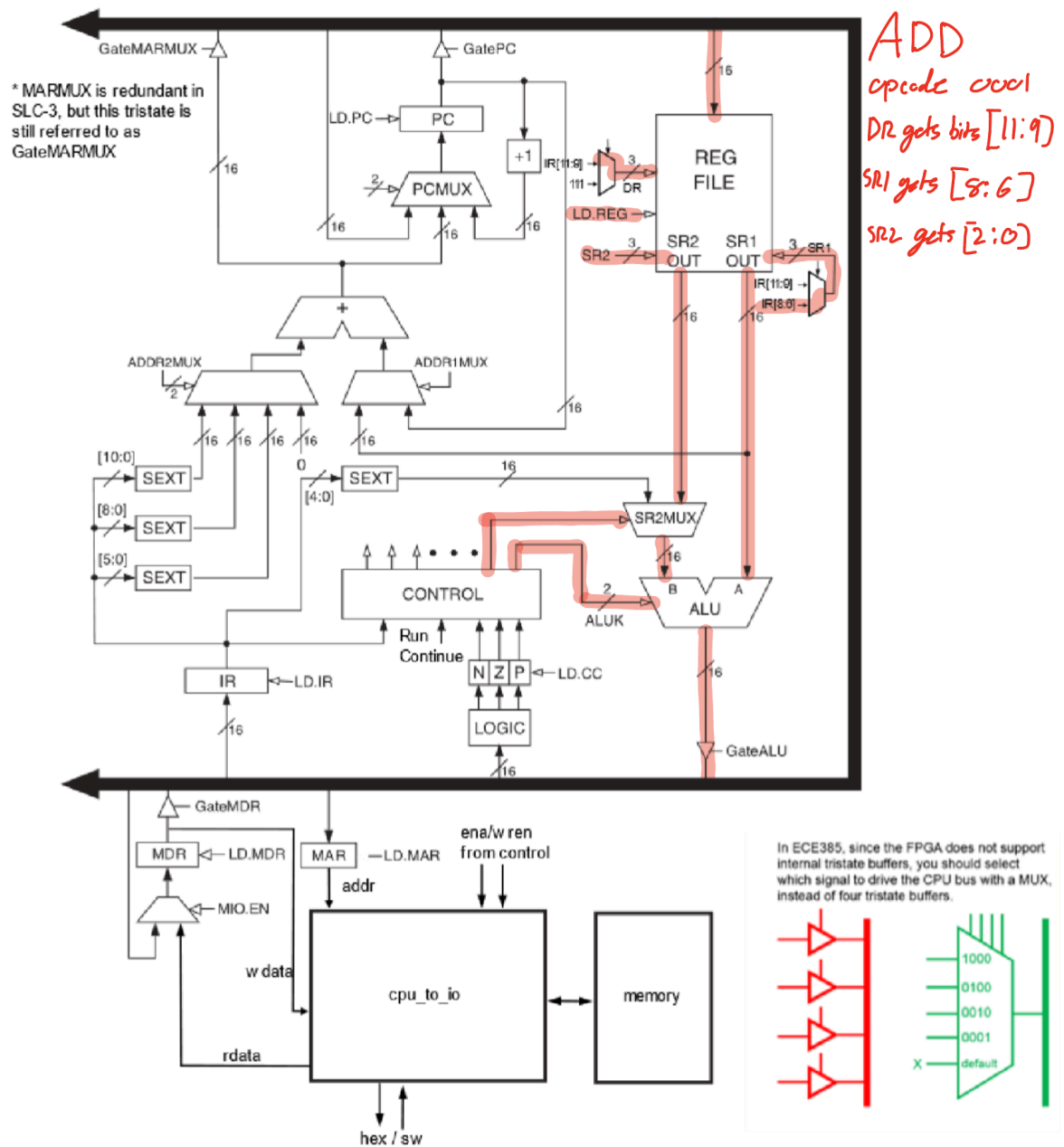


Figure 3 - ADD instruction datapath

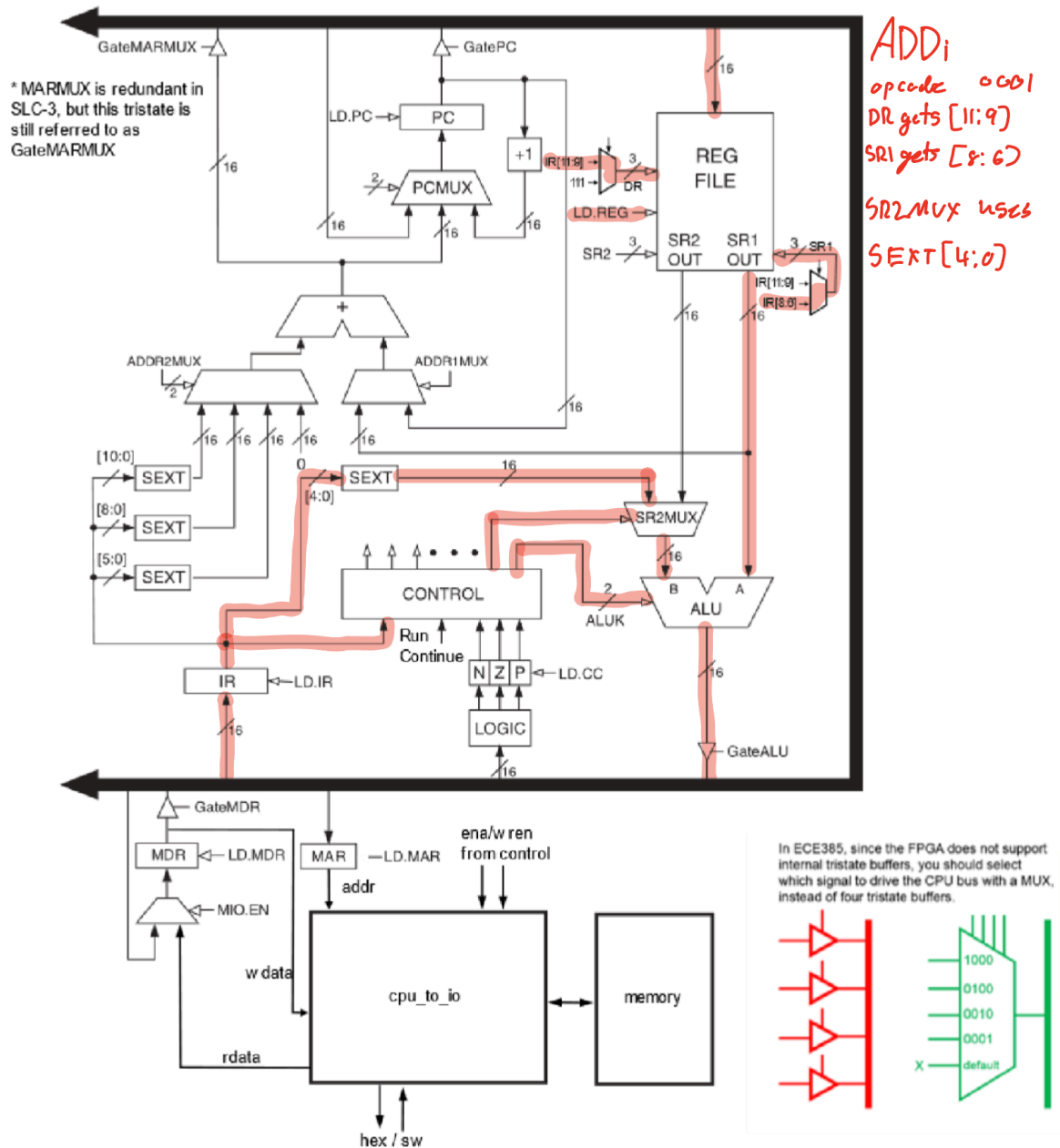


Figure 4 - ADDi instruction datapath

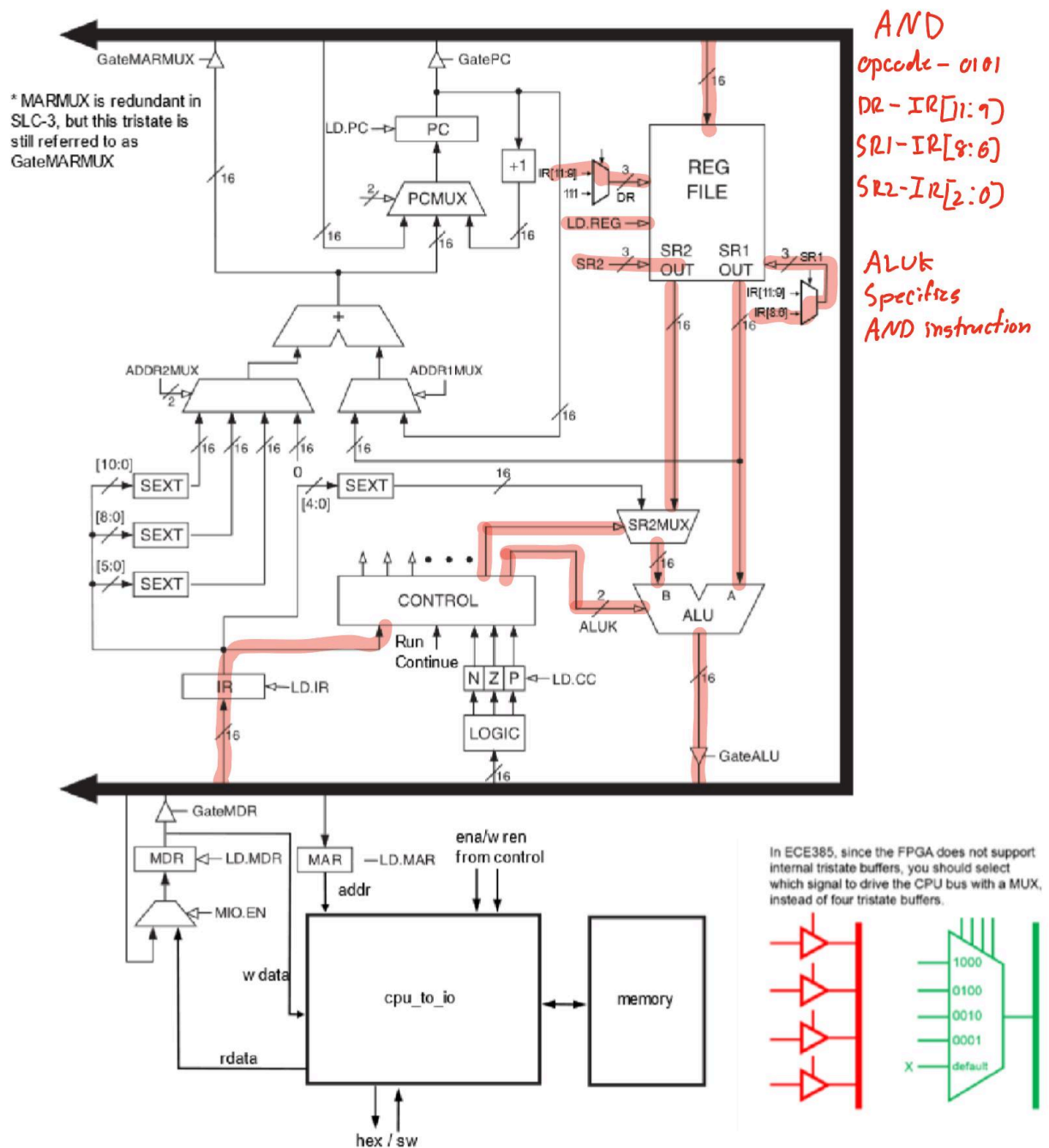


Figure 5 - AND instruction datapath

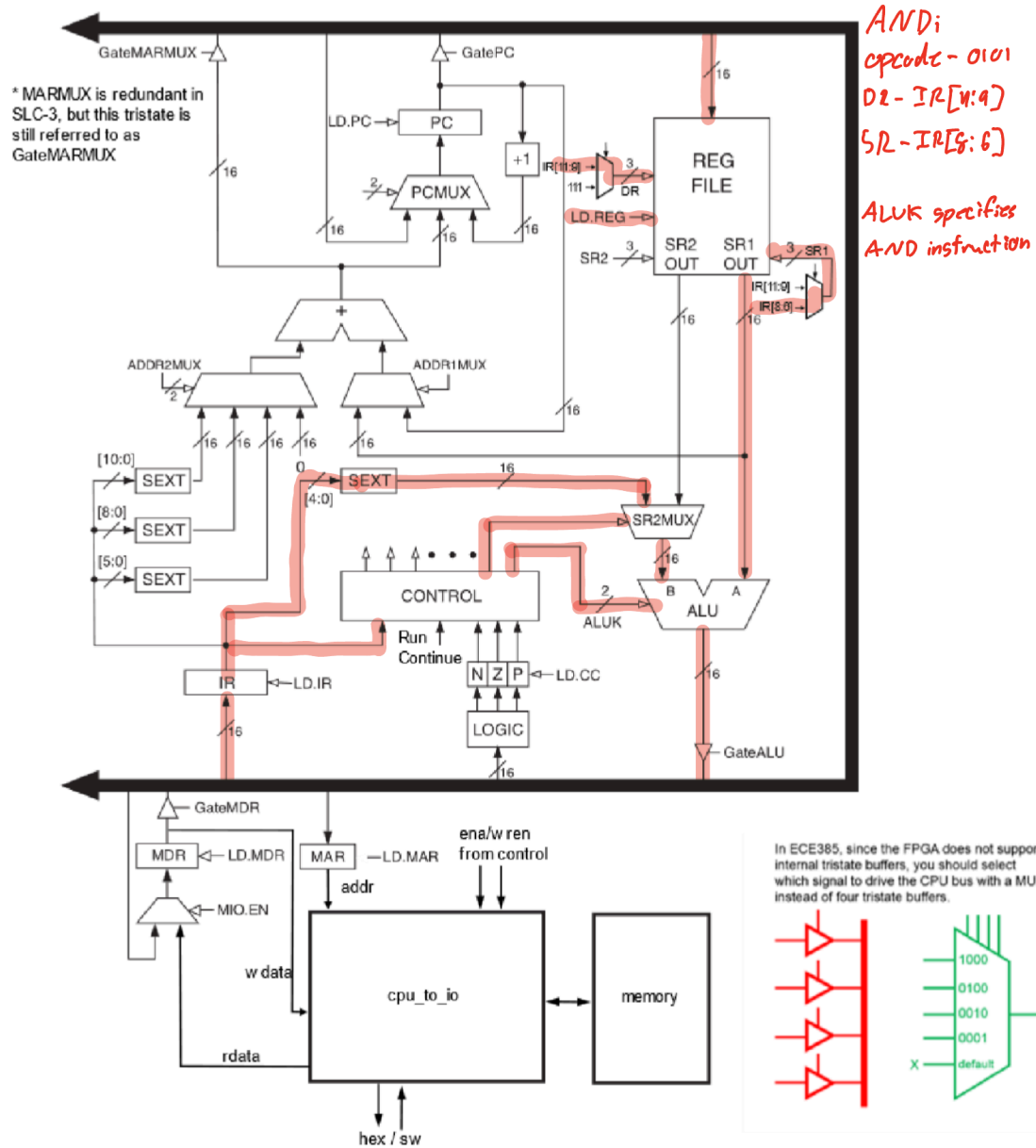


Figure 6 - ANDi instruction datapath

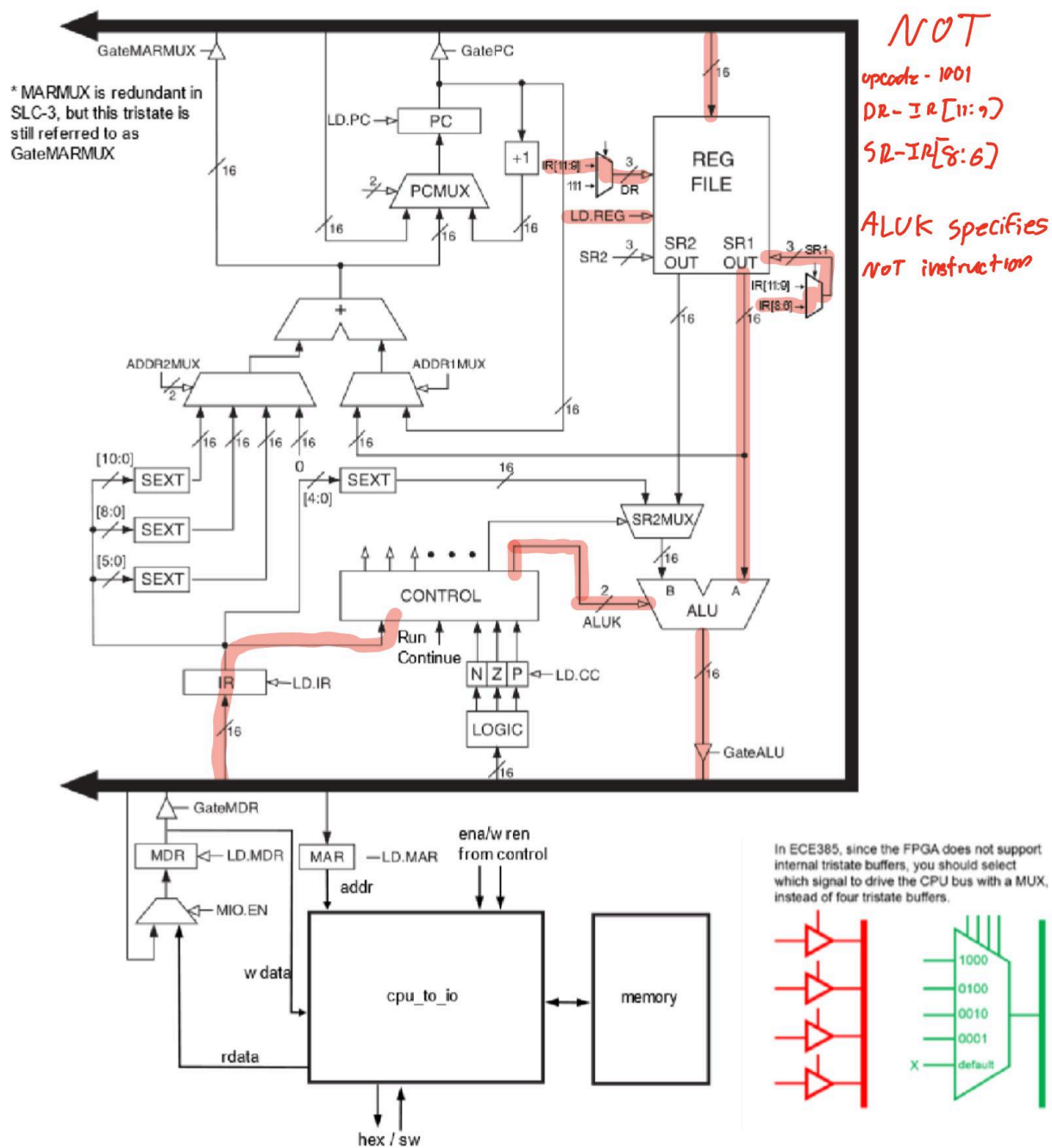


Figure 7 - NOT instruction Datapath

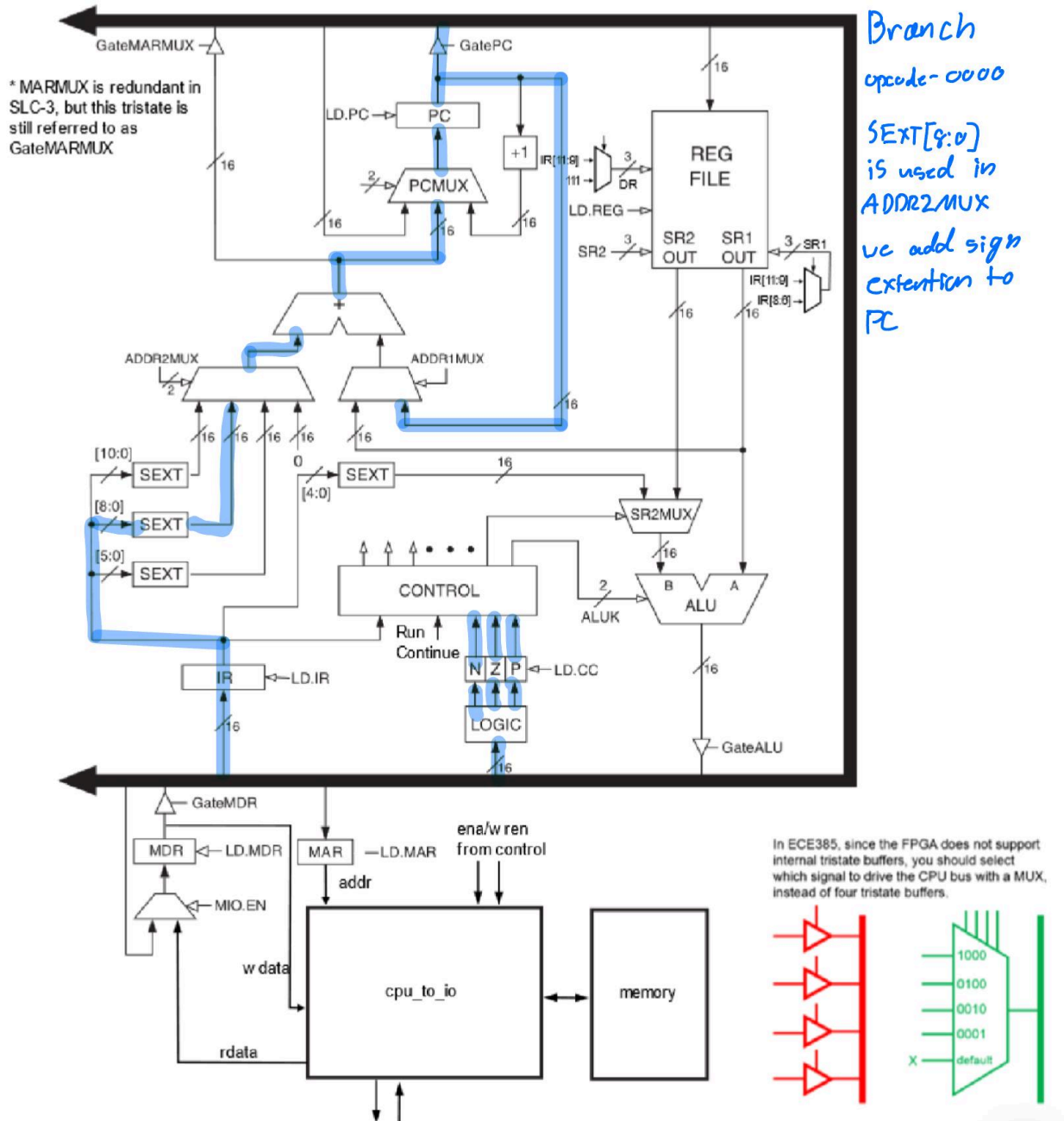


Figure 8 - BR (Branch) instruction datapath

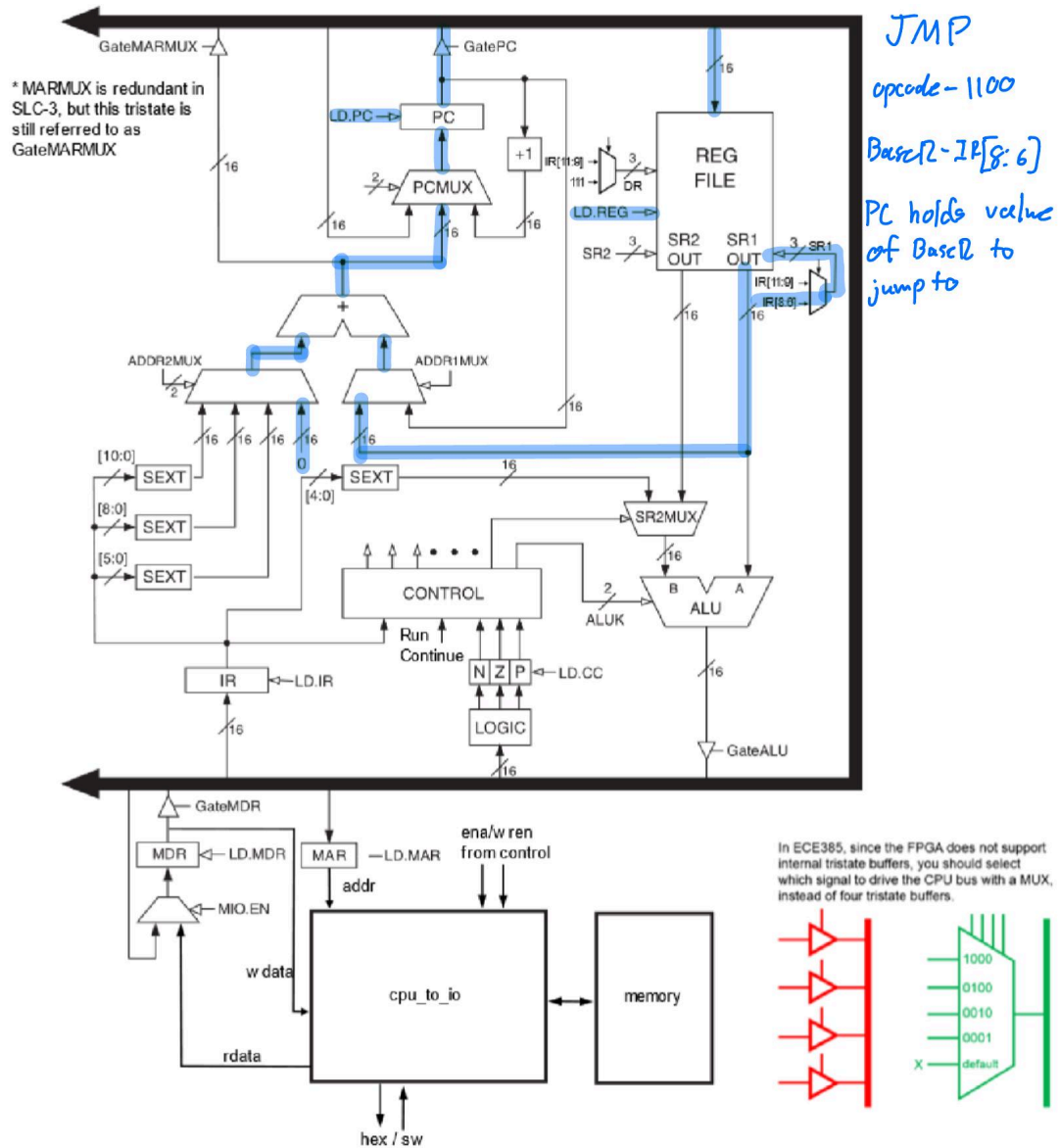


Figure 9 - JMP instruction datapath

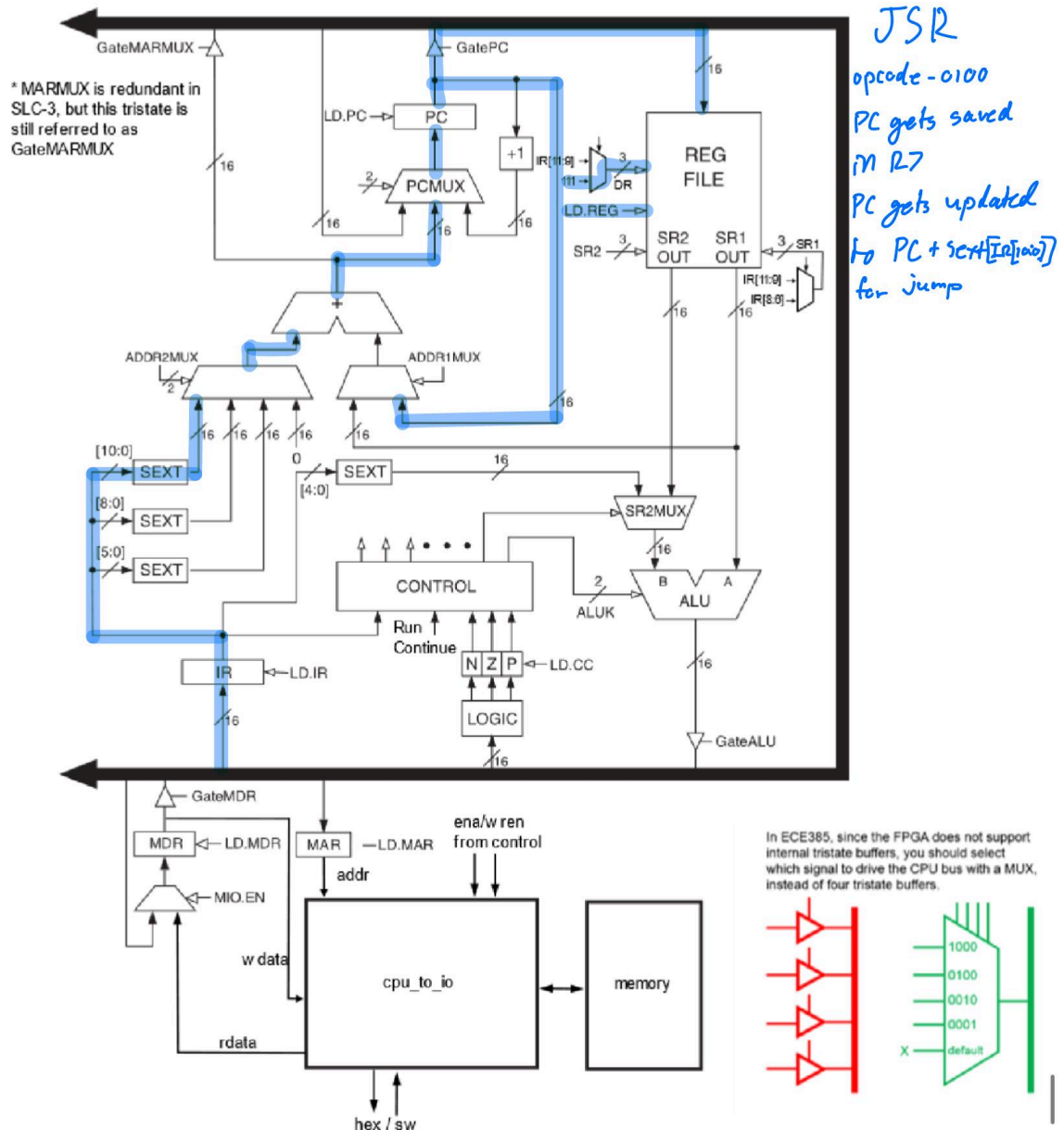


Figure 10 - JSR instruction datapath

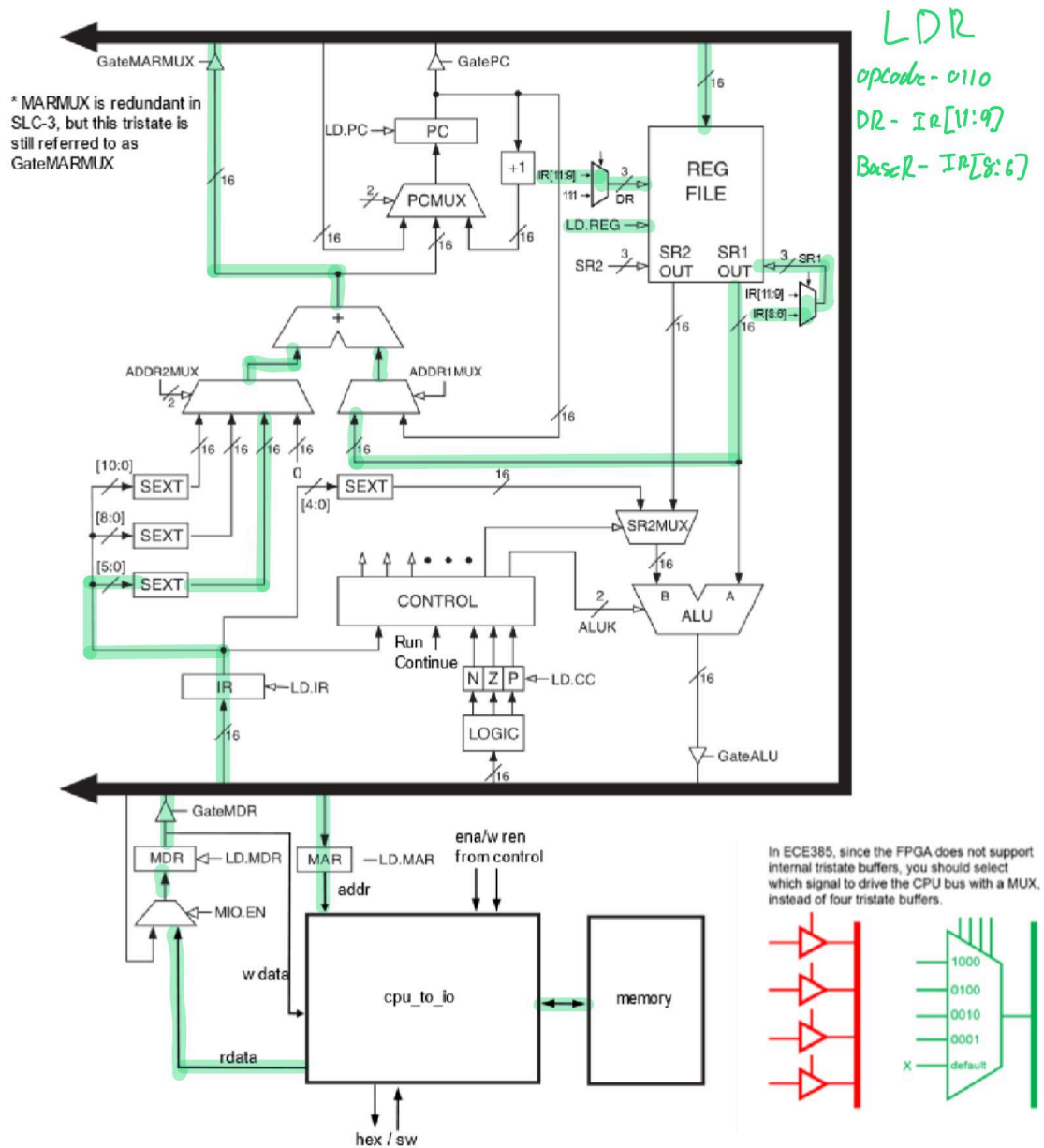


Figure 11 - LDR instruction datapath

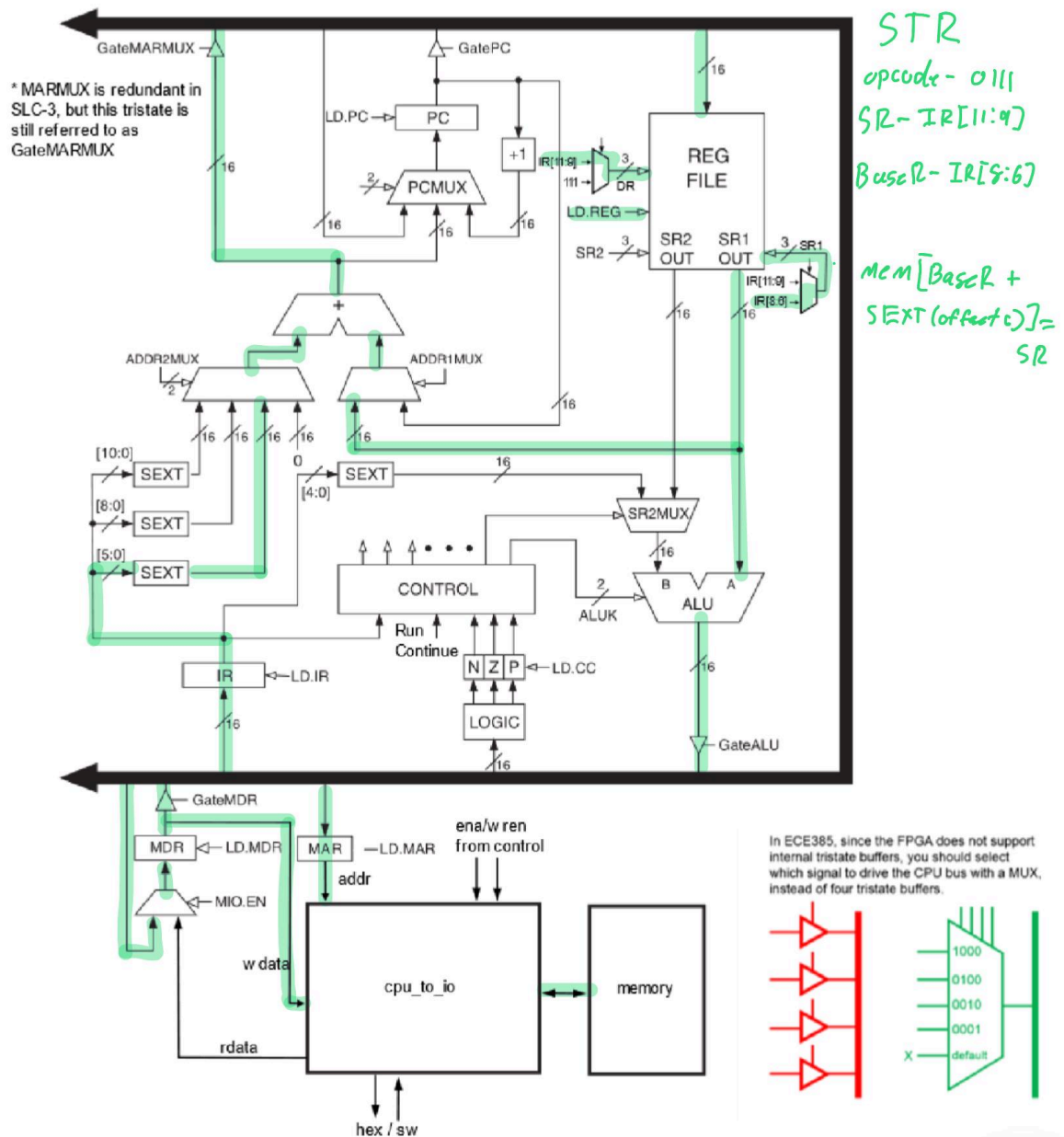


Figure 12 - STR instruction datapath

e. Block diagram of the top-level (processor_top.sv), which includes connections between the CPU, as well as the CPU_to_io, memory, and other modules.

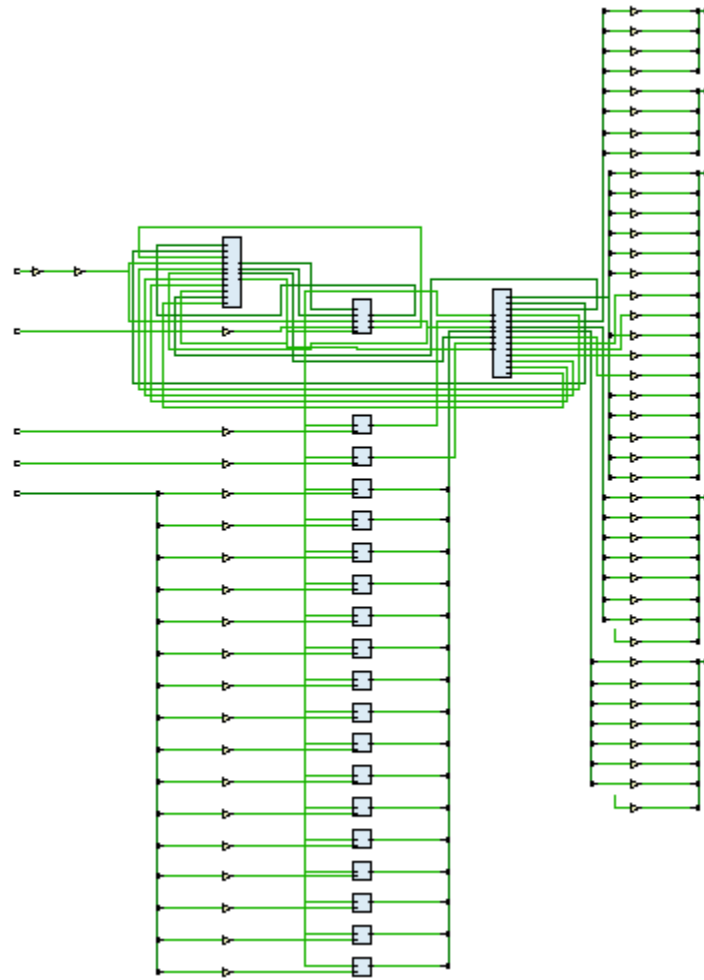


Figure 13 - processor top level schematic

f. Written Description of all .sv modules

Module: processor_top.sv (Top Level Module)

Inputs: [15:0] sw_i, clk, reset, run_i, continue_i

Outputs: [15:0] led_o, [7:0] hex_seg_left, [3:0] hex_grid_left, [7:0] hex_seg_right, [3:0] hex_grid_right

Description: This is the top level of the entire processor. It specifies the memory (BRam), SLC3 CPU, and the input/output features used (i.e. LEDs and Switches).

Purpose: In this module, we set up all the wiring to all of our subsystems which make up the entire system. The main connections are the I/O, the memory, and the general CPU design.

Module: slc3.sv

Inputs: clk, reset, run_i, continue_i, [15:0] sw_i

Outputs: [15:0] led_o, [7:0] hex_seg_o, [3:0] hex_grid_o, [7:0] hex_seg_debug, [3:0] hex_grid_debug, [15:0] sram_rdata, [15:0] sram_wdata, [15:0] sram_addr, sram_mem_ena, sram_wr_ena

Description: This module includes all the I/O the CPU works with, along with reading and writing from memory. It creates the CPU-to-IO bridge, connecting the wires between both subsystems.

Purpose: This is another top-level type of module responsible for wiring all the smaller subsystems.

Module: CPU.sv

Inputs: clk, reset, run_i, continue_i, [15:0] mem_rdata

Outputs: [15:0] hex_display_debug, [15:0] led_o, [15:0] mem_wdata, [15:0] mem_addr, mem_mem_ena, mem_wr_ena

Description: The CPU has all the function capability of working with I/O, along with reading and writing to memory. We connected all the wiring between the various sub-modules in this module to make the entire data path. This is the data needed for each instruction to execute correctly.

Purpose: This module has instantiated the entire datapath for the instructions to run, so this module is vital for the operation of the SLC3 processor.

Module: control.sv

Inputs: clk, reset, ir, ben, continue_i, run_i, [15:0] bus_output

Outputs: ld_mar, ld_mdr, ld_ir, ld_pc, ld_led, mio_en, ld_cc, ld_reg, sr1_sig, addr1_muxsig, [1:0] addr2_muxsig, [1:0] aluk_sig, gate_pc, gate_mdr, gate_marmux, gate_alu, dr_muxsig, [1:0] pcmux, sr2_muxsig, mem_mem_ena, mem_wr_ena

Description: This module instantiates the entire FSM and the order in which operations should take place in the CPU. We create all the states from the state diagram and set all of our control signals to become high or low depending on the operation that needs to be completed in each state. This is what allows the CPU to determine which bits must go where during an instruction. We run the fetch, decode, and execute cycle, specifying which state should happen after which.

Purpose: This module is fundamental to the CPU as we specify the FSM and order in which operations must be executed. This is what allows large programs to run in the correct order.

Module: sr1mux.sv

Inputs: sr1_sig, [2:0] ir119, [2:0] ir86

Outputs: [2:0] sr1_out

Description: A basic 2-1 mux that specifies if the instruction bits 11-9 or 8-6 should be used for source register 1. A 0 select results in IR119 signal going through mux and a 1 bit results in IR86 bits going through the mux.

Purpose: This allows us to determine which register source register 1 must have

Module: dr_mux.sv

Inputs: dr_muxsig, [2:0] ir119, [2:0] seven

Outputs: [2:0] dr_out

Description: This mux selects the destination register value from the IR bits. If the select for the mux is 0, then the destination register is IR119, if not then the destination register is 7.

Purpose: This mux is used to help choose the destination register value which is important and specified in all instructions.

Module: registers.sv

Inputs: [2:0] sr1, [2:0] sr2, [15:0] bus_value, ld_reg, [2:0] dr, clk, reset

Outputs: [15:0] sr1_out, [15:0] sr2_out

Description: This module is our register unit. We used unpacked arrays to make our register unit by specifying 8 registers. We specified that all these registers must be 0 during reset, otherwise, if we can load them, they will get their respective values from the operations being performed.

Purpose: The register unit is what allows us to hold values during instructions and get an output that can be used during computation.

Module: addr2mux.sv

Inputs: [1:0] addr2_select, [15:0] ir

Outputs: [15:0] addr2_muxout

Description: This module is a 4-1 mux used for sign-extending specified bits from the data bus. The select bits 00 output all 0's. The select bits 01 sign extends bits from IR[0:5], bits 10 extend IR[8:0], and bits 11 extend IR[10:0]. This output is used in further computation.

Purpose: The purpose of this module is primarily to sign extending the inputs we have from our data bus. Once we know how many bits to sign extend, we do this and use that number in other modules.

Module: addunit.sv

Inputs: [15:0] addr1_input, [15:0] addr2_input

Outputs: [15:0] add_out

Description: This module is an adder module that only computes the addition of our two 16-bit inputs. Add_out will always be the value of addr1_input and addr2_input if the path utilizes this module.

Purpose: The addunit is utilized to add an offset to the Program Counter (PC) when performing certain instructions, or to change the value of our address pointer due to our base register. There are several offsets we can add, which are determined by the ADDR2MUX in our diagram. The main purpose of this module is to add an offset to either the BaseR value or our PC, performing JMP and BR instructions.

Module: sr2mux.sv

Inputs: [15:0] sr2out, [15:0] ir

Outputs: [15:0] sr2_muxout

Description: This module determines whether we utilize Source Register 2, or an immediate value to perform our computation, based on a specific select bit, which in this case is ir[5], the 6th bit from the right of our Instruction Register. Sr2mux is a multiplexer that takes in the 6th bit of the instruction register and determines what value to use for the computation.

Purpose: Whenever we need to perform a specific computation through our ALU, we have two options for the ADD and AND operations: 1) utilizing Source Register 1 and Source Register 2. 2) Utilizing Source Register 1 and an Immediate number value. The purpose of this module is to allow us to figure out whether we want to utilize two registers, or other inputs for our ALU computation.

Module: alu.sv

Inputs: [1:0] aluk, [15:0] Aval, [15:0] Bval

Outputs: [15:0] computation

Description: This module performs the computation based on a specified function in our instruction register. The ALU module is a multiplexer that takes in a 2-bit select, aluk, and performs either ADD, AND, NOT, or PASS A.

Purpose: This module is important when we need to perform a specific computation based on our datapath. For example, the ADD and AND functions pass through this ALU module, with output computation as our final answer. Whether we utilize two registers or an immediate value is determined by the SR2 mux specified earlier.

Module: nzp_module.sv

Inputs: [15:0] busdata, clk, reset, ld_ben, ld_cc, [15:0] ir

Outputs: benout

Description: This module performs an NZP (negative, zero, positive) check on the data in the bus of this design. It utilizes two registers, one 3-bit register for NZP, and 1

single-bit register for the value of Branch Enable (ben). Both of these registers are controlled by a clock and its respective load signals we pass through from the control unit. We are given an equation to check for the ben signal, which determines the output of this module.

Purpose: The NZP module is crucial when doing the branch (BR) instruction, and this is conditional on the bus value. Based on a specified instruction (e.g. BRn, or BRnz, etc.), the program would branch off to a specific instruction if the condition is met. We know that this condition is met if nzp_module produces a 1 for ben, which tells the design that it needs to perform a branch operation.

Module: bus_mux.sv

Inputs: gatepc, gatemdr, gate_marmux, gate_alu, [15:0] pc, [15:0] mdr, [15:0] marmux, [15:0] alu

Outputs: [15:0] bus_out

Description: This module represents the main bus in our design, which is essential to transmit data from one module to another. The bus module partially acts as a mux, as we have 4 select signals, each representing one of the gates in our design: gatepc, gatemarmux, gatealu, and gatemdr. Only one of these signals can be high at once, which deploys our necessary data onto the bus.

Purpose: The bus module acts as a connector between all of our other submodules, transmitting data from one area to another. For example, when we need to transmit data after computing our operation, back to our destination register, we need to utilize the bus to store our value and use this value within the Destination Register. The bus module determines what value to put on the bus based on which gate is high, and this allows the design to transmit data from one place to another.

Module: pc_mux.sv

Inputs: [15:0] Din1, [15:0] Din2, [15:0] Din3, [15:0] Din4, [1:0] sel

Outputs: [15:0] Dout

Description: This module represents the PC mux in our design, which determines what value to utilize for our PC, or program counter. Din1 represents the PC + 1 value, which is the incremented value for PC, pointing to the next instruction. Din2 represents the output of the two addresses, and Din3 represents the value on our bus. Din4 is

hardwired to 0 as this mux only asks for 3 inputs. Each of the inputs in this module is important to determine the value of our PC, based on specified criteria.

Purpose: The purpose of this module is to keep track of the input of our PC register, which holds the value of our program counter. This multiplexer is dependent on several inputs, along with a PC select passed through from the control unit. Furthermore, the pcmux determines how we want to utilize our PC, and where in the program we want to execute it.

Module: two_one_mux.sv

Inputs: [15:0] D2in1, [15:0] D2in0, sel2

Outputs: [15:0] D2out

Description: This module is solely a 2 to 1 multiplexer, that takes in two 16-bit inputs, and a one-bit select bit. This select bit determines which input to choose as our output.

Purpose: This module is utilized as part of the “MIO mux” which is connected to our MDR. In this case, we have two inputs, the bus value, and the value from memory, and this multiplexer helps us decide which value we want to write to our MDR or the Memory Data Register. This is the only place where we use this specific MUX, as all other modules have their own ‘versions’ implemented.

Module: load_reg.sv

Inputs: clk, reset, load, [DATA_WIDTH-1:0] data_i

Outputs: [DATA_WIDTH-1:0] data_q

Description: This module represents a load register of variable bit sizes. We are able to set the size of this register as needed and as per our implementation, which allows us to only have one load register module instantiated in several areas within our design. The register is updated at each clock cycle and has its respective load signal that tells the register when to load a new/another value.

Purpose: This module is utilized in several areas of our design, especially within the NZP module and CPU unit. Each of our registers, the MDR, MAR, PC Register, and IR in our CPU module utilize the load_reg with their respective bit lengths. Furthermore, this module is utilized to store our NZP (negative, positive, zero) module values, as these need to be updated each time with different loads. The load_reg module allows us to keep track of our data, and visualize our outputs to ensure proper data storage.

Module: test_memory.sv

Inputs: clk, reset, [15:0] data, [9:0] address, ena, wren

Outputs: [15:0] readout

Description: This module is 'simulation memory' which we can utilize in place of the physical memory in our simulation. For this to work, we are provided with a types.sv file with the other instructions to perform each test. This memory is utilized when performing simulations of our design.

Purpose: The purpose of this module is to allow us to debug and test our design as we progress through it. By testing each module using this memory, we can read out the output and understand if the work we are doing is correct. The test memory is crucial in the debugging process of this design, producing the desired outputs when reading out waveform, based on file types.sv.

Module: memory.sv

Inputs: clk, reset, [15:0] data, [9:0] address, ena, wren

Outputs: [15:0] readout

Description: This module is responsible for handling the operation we utilize the memory for within our design. In each state, we define several signals that tell our system to perform specific actions to memory, which are passed on here are the 'wren' signal. This is the read and write enable, which needs to be 1 to access this memory.

Purpose: The purpose of the memory module is to handle the read and write operation for each state, as we define it. For example, when we read or write from our MAR, we utilize the memory to perform this process, so this is a crucial part of our design.

Module: cpu_to_io.sv

Inputs: clk, reset, [15:0] cpu_addr, cpu_mem_ena, cpu_wr_ena, [15:0] cpu_wdata, [15:0] cpu_rdata, [15:0] sram_addr, sram_mem_ena, sram_wr_ena, [15:0] sram_wdata, [15:0] sram_rdata, [15:0] sw_i

Outputs: [3:0] hex_grid_o, [7:0] hex_seg_o

Description: This module is responsible for being the interface between the computer, and the memory input and output. The CPU_TO_IO manages the input and output devices, which are the display and the switches in this case. When the memory address is 0xFFFF, this tells the CPU_TO_IO module to take input from our switches, further building on the SLC-3's memory-mapped IO.

Purpose: This module allows us to correctly load the values of our IR through our switches. Furthermore, it allows this computer to work with the input and output devices, both displaying and reading the external input and outputs.

Module: `instantiate_ram.sv`

Inputs: reset, clk

Outputs: [9:0] addr, [15:0] data, wren

Description: This module instantiates the RAM on our Urbana Board, allowing us to utilize memory functions such as read and write. This is necessary to access the memory in any way, as we need to first instantiate it to be able to use it.

Purpose: For us to write and read to memory in our operations, we first need to instantiate our memory. This is important to store our data and access it in any way during each state.

Module: `sync.sv`

Inputs: clk, d

Outputs: q

Description: The sync module is designed as a debouncer circuit, along with a synchronizer for the pushbutton and switch inputs. As this utilizes a D-flip flop, we have two inputs, clock, and d, along with output q, which is the next state variable for the flip-flop.

Purpose: Its purpose is to reduce as much noise as possible from the switches we utilize. On a high level, the synchronizer module reduces noise after each switch or button flip and ensures a stable output through q and the next state variables.

Module: `hex_driver.sv`

Inputs: clk, reset, [3:0] in[4]

Outputs: [7:0] hex_seg, [3:0] hex_grid

Description: The Hex Driver module converts each nibble into a corresponding pattern and then keeps processing the bits to ensure the correct output. These inputs are based on switches we ultimately use to load and then execute our operations. Each bit in the Hex Driver is tied to a display bit, and the code converts our switch binary values to hexadecimal, allowing us to see outputs in the form of numbers 0-9 and letters A through F. It translates each nibble into the correct hex representation and ultimately outputs them onto our Urbana board display.

Purpose: The Hex Driver module is responsible for displaying our output on the hex display of our Urbana Board. It takes in several inputs, including Clock, Reset, and 4-bit In. From this, it outputs an 8-bit hex seg and 4-bit Hex_grid.

g. Description of the operation of the control unit

i. Named control.sv, this is the control unit for the SLC-3. Describe in words how the control unit controls the various components of the SLC-3 based on the current instruction.

The control unit is responsible for the main executions of the SLC-3 processor. From our datapath, we can identify various control signals, such as LD.CC, LD.REG and several others tell each component to either load or select values based on a multiplexer. In our control unit, we have listed each state, along with its respective next state, and current state outputs, which dictates what signals should be high, and which should be low, along with the select bits of a multiplexer if we have to utilize one in that specific state. For example, in our ADD state, we need our ALUK signal to correspond to “add” for computation mux to give us the desired output. Based on each respective signal, the control unit controls each of the components in the SLC-3, either keeping it on, off, or with a specific select value.

h. State Diagram of Control Unit

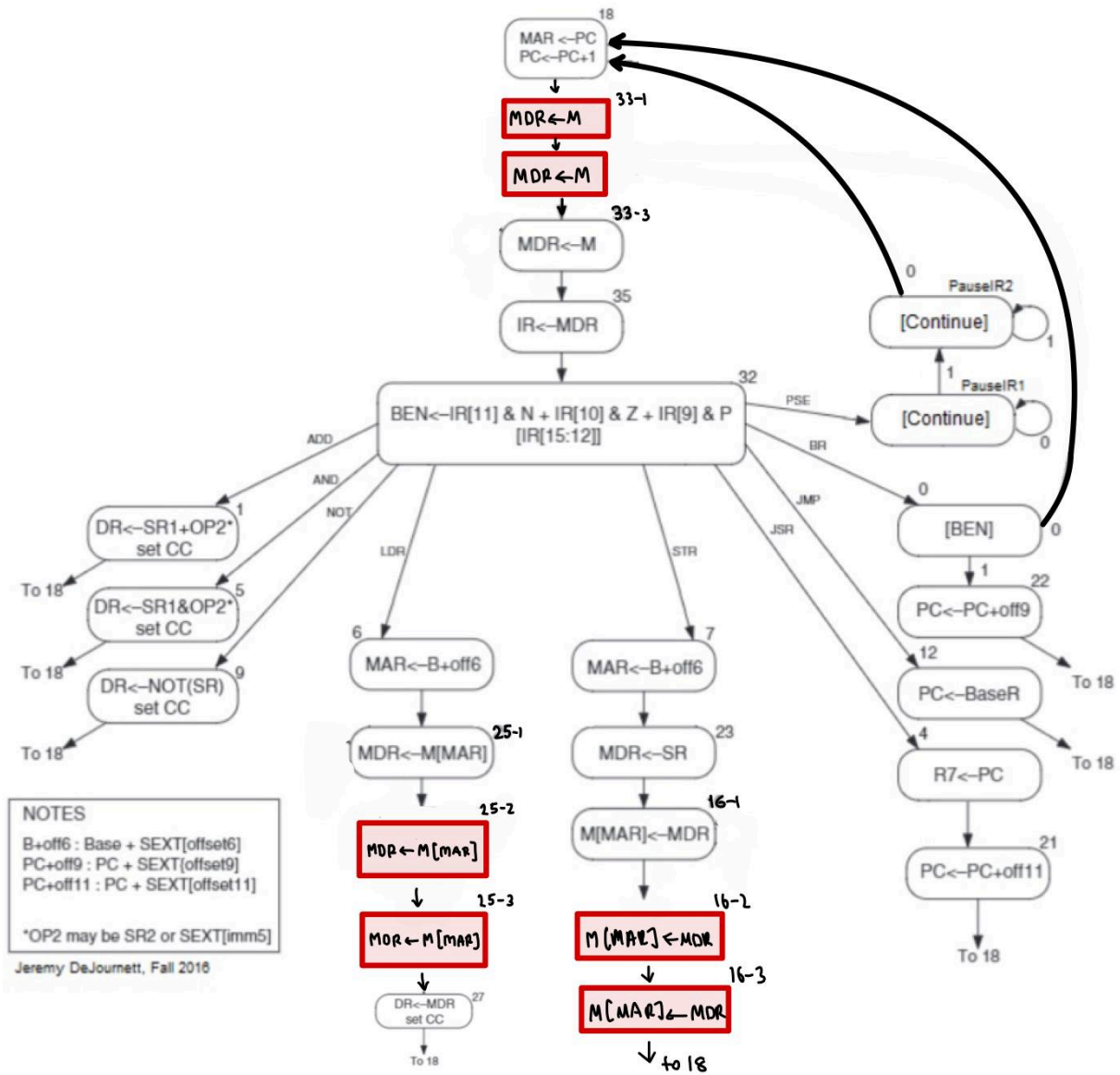


Figure 14 - Full Diagram of the Control Unit (Updated from Experiment Guide)

- a. Simulate the completion of all 7 test programs, I/O Test 1, I/O Test 2, Self-Modifying Code, XOR, Multiplier, Auto Count, and Sort.

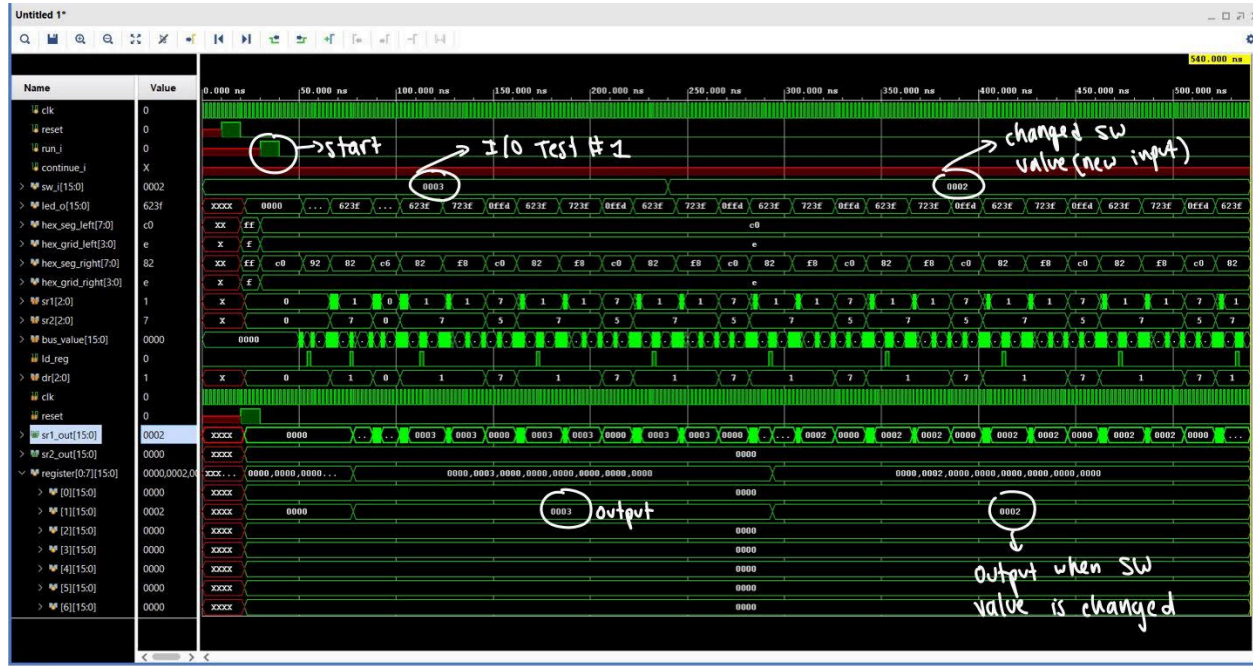


Figure 15 - Annotated Simulation of I/O Test 1



Figure 16 - Annotated Simulation of I/O Test 2

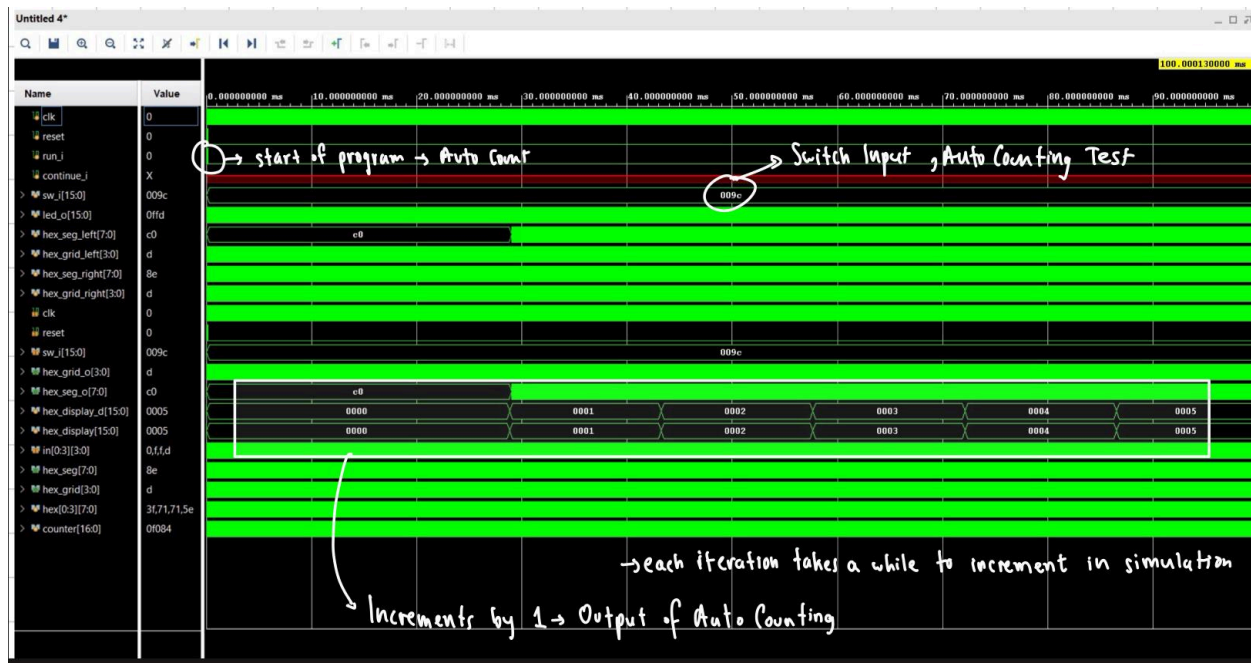


Figure 17 - Annotated Simulation of Auto Counting Test

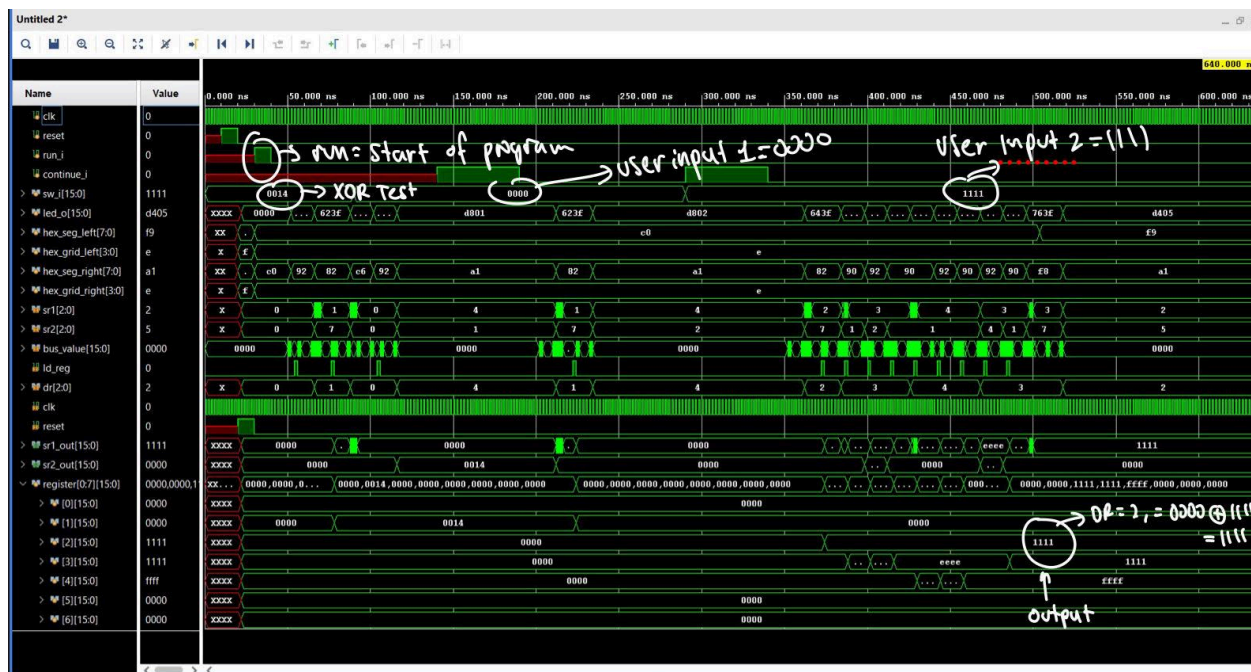


Figure 18 - Annotated Simulation of XOR Test

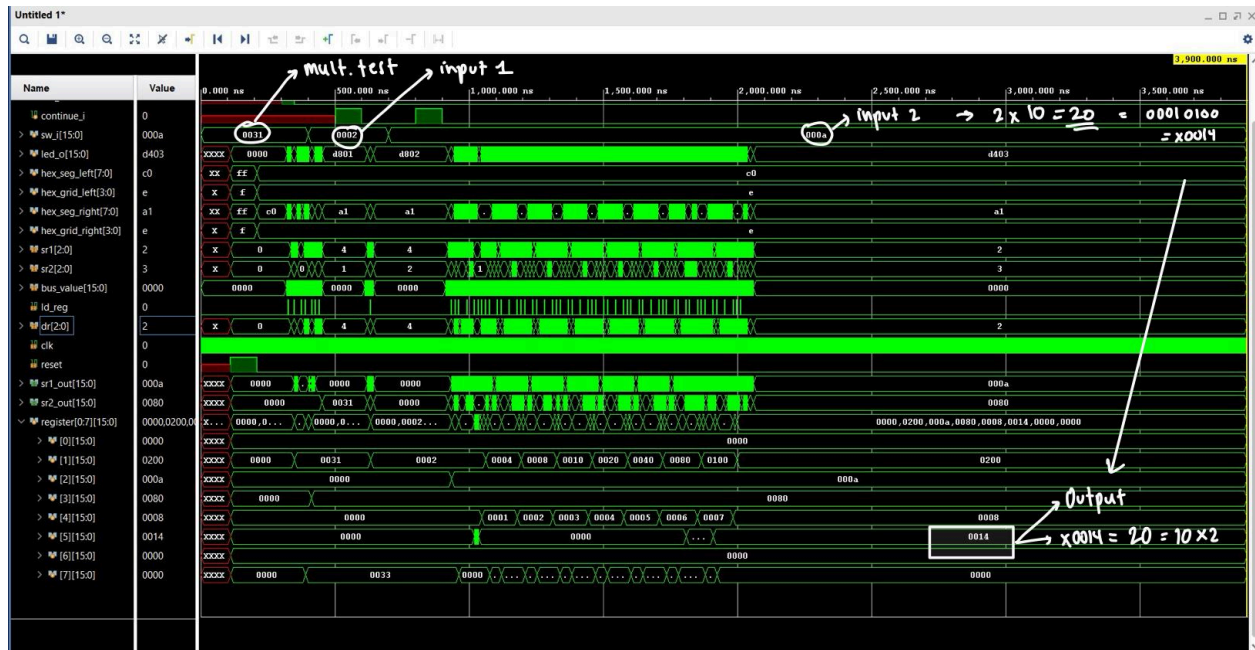


Figure 19 - Annotated Simulation of Multiplication Test

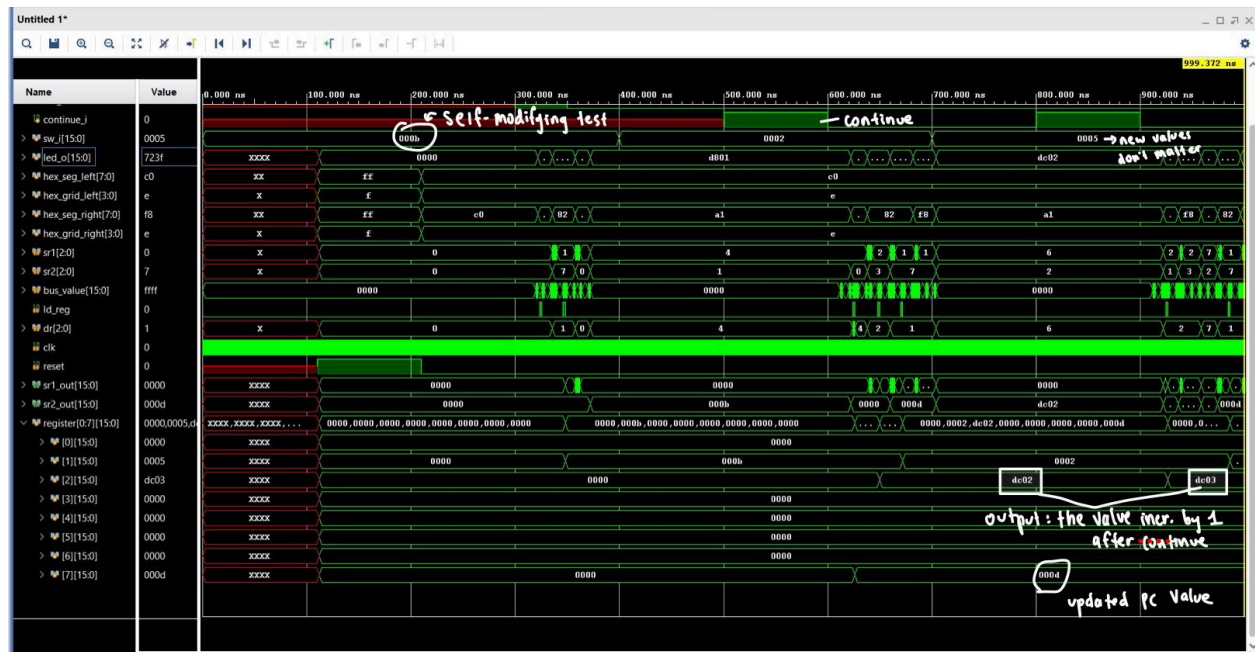


Figure 20 - Annotated Simulation of Self-Modifying Test

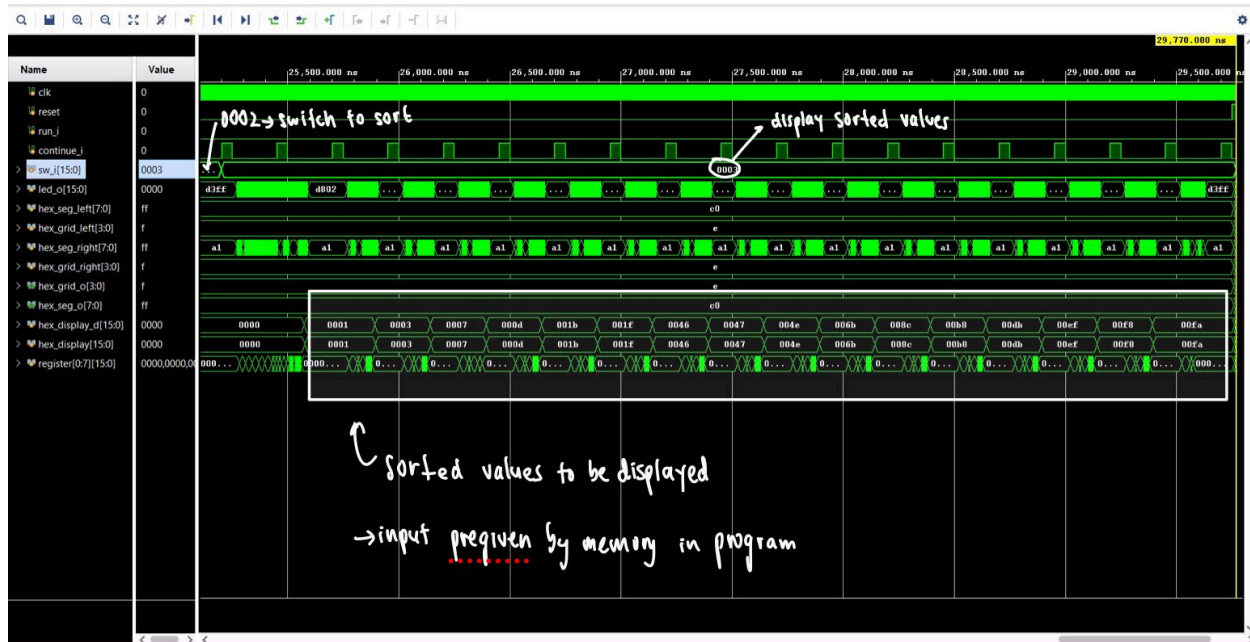


Figure 21 - Annotated Simulation for the Sort Test

4. Post-Lab Questions

a. Fill out the Design Resources and Statistics table.

LUT	414
DSP	0
Memory (BRAM)	1
Flip-Flop	344
Latches	0
Frequency	$1/(10-0.463) = \text{WNS} = 104 \text{ MHz}$
Static Power	0.074 W
Dynamic Power	0.010 W
Total Power	0.085 W

What is CPU_TO_IO used for, i.e., what is its main function?

CPU_TO_IO manages the Urbana Board's I/O devices, which in this case are the displays and the switches. Both of these devices share the same memory address, as one is only input, and the other is only output. For example, both the Hex Display and Switches share memory address 0xFFFF. This is used to take our input from the switches, which we utilize when determining what tests and instructions to run. As our I/O is memory-mapped for the SLC-3, we need a way to access and display the values in memory, which is where CPU_TO_IO plays a key role by managing all of the board's physical input and output devices.

What is the difference between BR and JMP instructions?

We know that both BR and JMP instructions modify our PC value, based on the passed inputs, which can include an immediate value for JMP or an offset added to the PC in the BR function. Both BR and JMP perform similar actions, as they both modify our Program Counter (PC). When we call a BR in LC-3 assembly, this is conditional on the bus value, as based on the provided conditions, we need to check if the value is negative, positive, or zero, as indicated by the NZP module in our design. When we add the offset to our PC+1, then we go to a specific PC we desire. However, in JMP, we unconditionally jump to a desired PC based on the passed input. For JMP, our target address is specified by a register, and this is always taken without an NZP check like the BR instruction.

What is the purpose of the R signal in Patt and Patel? How do we compensate for the lack of the signal in our design? What impact does this have on performance?

In Patt and Patel, the R signal is essentially a "ready bit" which is used to indicate when we are ready to produce a result. This allows us to know when to proceed states, ensuring we do not continue without reading our results. In our design, we do not have a "ready bit" but rather two extra states that we treat as "wait" states. For example, states 33, 26, and 15 include a ready bit in the initial Patt and Patel LC-3 FSM. In our version of SLC-3, we add two extra states for each of them, as we know that it takes 2 clock cycles for our ready bit to be "ready." The two states act as an execution delay rather than including a ready bit specified by Patt and Patel. However, this does come with an impact on performance, as there is an extra delay for three states in our state diagram. One impact could be less efficiency, as we have determined two states of waiting. However, in the situation that the ready bit completes its process earlier, there would be additional disregarded delay. Furthermore, there is an overall delay in the system, as we have to wait for the extra states several times, reducing the speed of the SLC-3 by a few clock cycles for each execution.

5. Conclusion

Our design worked quite well in our demo, as we were able to make each module its entity while developing our Hardware Description in Vivado. This made it much easier to debug, but also a lot easier to connect each module in our CPU unit. Overall, we were able to get every test working, with several debugging issues, which included wrong bit sizes, wrong input and outputs, and incorrect naming, which we were able to debug through the waveform. Understanding the logic and mapping out each datapath before connecting the components allowed us to make the module connection a lot easier, ultimately leading to minimal debugging for this experiment.

The instructions were quite good for this lab, as it was not too difficult to understand the goals. As we had an idea of the frameworks from previous labs, understanding the control unit, and other modules was quite straightforward, which we believe should be kept in the future.